MORSE CODE TRANSLATOR & LOGGER

Console-Based C Application — Group Project Proposal

Bridging Classical Communication with Modern Software Engineering

Course	Introduction to Programming / C Programming
Project Type	Group Project (5 Members)
Application Type	Console-Based C Application
Domain	Communication Technology / Embedded Systems
Budget Implication	Software only — no hardware cost
Date Prepared	May 2026

This document presents the full project proposal for a Morse Code Translator & Logger system to be developed in C as a 5-member group project. It covers system overview, module breakdown, structure definitions, function lists, work distribution, and alignment with all assignment requirements.

1. Project Overview

The **Morse Code Translator & Logger** is a console-based C application that provides a complete, multi-functional platform for encoding and decoding Morse code, managing session history, supporting multiple alphabets, and producing structured reports. Morse code — originally invented for telegraph communication in the 1830s — remains relevant today in aviation, amateur radio (ham radio), emergency signalling, and accessibility technology for individuals with motor disabilities.

This system models a real-world communication-assistant tool used by radio operators, military communication trainees, and accessibility engineers. It is not a toy demo; it solves the genuine problem of translating and archiving human-readable messages into Morse code (and vice-versa) with multi-alphabet support, session persistence, user management, and statistical analytics.

1.1 Real-World Relevance

- Ham radio operators translate call signs and messages to/from Morse daily.
- Aviation: pilots still learn Morse for NDB (Non-Directional Beacon) identification.
- Emergency preparedness: Morse SOS (··· --- ···) is an internationally recognised distress signal.
- Accessibility: Morse-based AAC (Augmentative & Alternative Communication) devices assist users with ALS, cerebral palsy, and locked-in syndrome.
- Military training: many armed forces still include Morse in signal corps curricula.

1.2 System Capabilities (Summary)

Capability	Description
Text → Morse	Converts typed text into standard ITU Morse code strings
Morse → Text	Decodes Morse strings back to readable text with validation
Session Logging	Records every translation with timestamp, user ID, and mode
Multi-Alphabet	Supports Latin, Numeric, Punctuation, and Prosign alphabets
User Profiles	Manages user accounts with stats and session history
Analytics	Displays frequency tables, accuracy stats, and session summaries
Search & Sort	Searches session logs; sorts by date, user, or message length
Export Report	Generates a plain-text report file of a session

2. Module Breakdown & Member Responsibilities

The system is divided into 5 independent but interconnected modules. Each module is owned by one group member, contains at least one structure and at least 5 logical functions, and integrates with the others through well-defined interfaces.

M1 | Encoder / Decoder Engine (Member 1)

Description	Core translation logic. Converts text to Morse and Morse to text. Maintains the master code table for all supported alphabets.
Structure	MorseEntry — holds character, Morse code string, alphabet type, frequency count.

Key Functions	
`loadMorseTable()`	initialise the full ITU Morse table into an array of MorseEntry
`encodeText()`	encode a full sentence to Morse, word by word
`decodeMorse()`	decode a Morse string back to text
`validateMorseInput()`	check that input contains only '.', '-', '/', and spaces
`lookupCharacter()`	binary-search the sorted table for a given character
`getMorseForChar()`	return the Morse string for a single character
`getCharFromMorse()`	reverse lookup: Morse string → character

M2 | Session Logger (Member 2)

Description	Records every encode/decode operation as a timestamped log entry. Supports saving logs to file and reloading them on startup.
Structure	SessionLog — session ID, user ID, timestamp string, input text, Morse output, mode (E/D), duration.

Key Functions	
`createSession()`	initialise a new SessionLog with auto-incremented ID
`logTranslation()`	append a completed translation to the in-memory log array
`saveLogsToFile()`	persist all session logs to a .txt flat file
`loadLogsFromFile()`	re-read logs from file into the array on program start
`displaySessionHistory()`	print all logs in a formatted table view
`filterLogsByUser()`	return only sessions belonging to a given user ID
`clearOldLogs()`	delete logs older than N days

M3 | User Profile Manager (Member 3)

Description	Manages user accounts: registration, login, password validation, and per-user statistics (total translations, accuracy, favourite alphabet).
Structure	UserProfile — user ID, username, hashed PIN, total encodes, total decodes, preferred alphabet, registration date.

Key Functions	
`registerUser()`	create a new UserProfile, validate unique username
`loginUser()`	authenticate by username + PIN
`displayUserStats()`	print a user's lifetime translation statistics
`updateUserStats()`	increment encode/decode counters after each operation
`searchUserByName()`	linear search through users array by username
`sortUsersByActivity()`	sort users array by total translation count (bubble sort)
`deleteUser()`	remove a user profile and archive their sessions

M4 | Multi-Alphabet Manager (Member 4)

Description	Manages distinct alphabet sets: Latin letters, digits, punctuation, and Prosigns (procedural signals like AR, SK, BT). Allows switching active alphabet at runtime.
Structure	AlphabetSet — alphabet ID, name string, array of MorseEntry, entry count, enabled flag.

Key Functions	
`initAlphabets()`	load all four alphabet sets at startup
`switchAlphabet()`	change the active alphabet used for translation
`displayAlphabetTable()`	pretty-print the full table of an alphabet set
`addCustomEntry()`	allow a user to add a custom character-Morse pair
`removeCustomEntry()`	remove a previously added custom entry
`mergeAlphabets()`	combine two alphabet sets for mixed-mode translation
`validateAlphabetCoverage()`	report which input characters are not in active set

M5 | Analytics & Report Generator (Member 5)

Description	Analyses session logs to produce frequency statistics, error reports, and plain-text report files. Provides search and sort over logs.
Structure	AnalyticsReport — report ID, generated-by user, date, total sessions, most active user, most used character, average message length, error count.

Key Functions	
`generateReport()`	build an AnalyticsReport from current session logs
`printReportSummary()`	display the report in a formatted console view
`exportReportToFile()`	write the report as a plain-text .txt file
`getMostUsedCharacter()`	scan logs and count per-character frequency
`sortLogsByDate()`	sort the session log array chronologically
`searchLogByKeyword()`	search for a keyword in all stored messages
`calculateErrorRate()`	compute the ratio of failed decodes to total attempts

3. Data Structures (C struct Definitions)

Below are the five primary C structures used in the system. Each is owned by one module and may be nested or passed to functions across modules.

MorseEntry (M1 — Encoder/Decoder)

```
typedef struct {
char character; /* The text character, e.g. 'A' */
char morseCode[12]; /* Morse string, e.g. ".-" */
int alphabetType; /* 0=Latin,1=Digit,2=Punct,3=Prosign */
int usageCount; /* How many times this char was encoded */
} MorseEntry;
```

SessionLog (M2 — Session Logger)

```
typedef struct {
int sessionID;
int userID;
char timestamp[20]; /* "YYYY-MM-DD HH:MM:SS" */
char inputText[256];
char morseOutput[1024];
char mode; /* 'E' = encode, 'D' = decode */
int durationMs;
int errorFlag;
} SessionLog;
```

UserProfile (M3 — User Manager)

```
typedef struct {
int userID;
char username[50];
char pin[7]; /* 6-digit PIN, stored as string */
int totalEncodes;
int totalDecodes;
int preferredAlphabet;
char regDate[12]; /* "YYYY-MM-DD" */
} UserProfile;
```

AlphabetSet (M4 — Alphabet Manager)

```
typedef struct {
int alphabetID;
char name[30]; /* e.g. "Latin", "Prosign" */
MorseEntry entries[128]; /* Nested array of MorseEntry */
int entryCount;
int enabled; /* 1 = active, 0 = disabled */
} AlphabetSet;
```

AnalyticsReport (M5 — Analytics)

```
typedef struct {
int reportID;
int generatedByUserID;
char date[12];
int totalSessions;
char mostActiveUser[50];
char mostUsedChar;
float avgMessageLength;
```

```
int totalErrors;
} AnalyticsReport;
```

4. Complete Function List (25 Functions)

The table below lists all 25 required functions (5 per member). Each function is described with its signature, owning module, and purpose.

#	Function Name	Module	Return / Params	Purpose
1	`loadMorseTable()`	M1	`void / AlphabetSet*`	Initialise full ITU Morse table
2	`encodeText()`	M1	`char* / char*, AlphabetSet*`	Encode sentence to Morse string
3	`decodeMorse()`	M1	`char* / char*, AlphabetSet*`	Decode Morse string to text
4	`validateMorseInput()`	M1	`int / char*`	Return 1 if input is valid Morse
5	`lookupCharacter()`	M1	`int / MorseEntry*, char, int`	Binary search for a character
6	`createSession()`	M2	`SessionLog / int, char`	Create a new session record
7	`logTranslation()`	M2	`void / SessionLog*, SessionLog`	Append log to array
8	`saveLogsToFile()`	M2	`int / SessionLog*, int, char*`	Persist logs to .txt file
9	`loadLogsFromFile()`	M2	`int / SessionLog*, char*`	Reload logs from file
10	`displaySessionHistory()`	M2	`void / SessionLog*, int`	Print formatted log table
11	`registerUser()`	M3	`int / UserProfile*, char*, char*`	Register new user
12	`loginUser()`	M3	`int / UserProfile*, int, char*, char*`	Authenticate user
13	`displayUserStats()`	M3	`void / UserProfile*`	Print user statistics
14	`updateUserStats()`	M3	`void / UserProfile*, char`	Increment encode/decode counter
15	`sortUsersByActivity()`	M3	`void / UserProfile*, int`	Bubble-sort by total translations
16	`initAlphabets()`	M4	`void / AlphabetSet*`	Load all four alphabet sets
17	`switchAlphabet()`	M4	`void / AlphabetSet*, int`	Set active alphabet by ID
18	`displayAlphabetTable()`	M4	`void / AlphabetSet*`	Print full alphabet/Morse table
19	`addCustomEntry()`	M4	`int / AlphabetSet*, char, char*`	Add user-defined entry
20	`validateAlphabetCoverage()`	M4	`void / char*, AlphabetSet*`	Report missing characters

#	Function Name	Module	Return / Params	Purpose
21	`generateReport()`	M5	`AnalyticsReport / SessionLog*, int`	Build analytics report
22	`printReportSummary()`	M5	`void / AnalyticsReport*`	Display report to console
23	`exportReportToFile()`	M5	`int / AnalyticsReport*, char*`	Write report to .txt
24	`sortLogsByDate()`	M5	`void / SessionLog*, int`	Chronological sort of logs
25	`searchLogByKeyword()`	M5	`int / SessionLog*, int, char*`	Keyword search in logs

5. Menu-Driven Interface Design

The main() function hosts a switch-based menu. Sub-menus are handled by dedicated functions called from each case. The design follows standard console UX patterns for C applications.

```
================================================
     MORSE CODE TRANSLATOR & LOGGER  v1.0
================================================
  Logged in as: [username]   Alphabet: [Latin]
------------------------------------------------
  MAIN MENU
  1. Encode Text  → Morse Code
  2. Decode Morse → Text
  3. View Session History
  4. Manage User Profile
  5. Alphabet Settings
  6. Analytics & Reports
  7. Logout / Switch User
  0. Exit
------------------------------------------------
  Enter choice: _
```

Each menu option calls the appropriate module function. Input is validated before processing. Invalid choices trigger a warning and re-display the menu.

6. Assignment Requirements Checklist

Status	Requirement	How It Is Met
✔	At least 5 structures	MorseEntry, SessionLog, UserProfile, AlphabetSet, AnalyticsReport — one per module
✔	At least 25 logical functions	5 functions × 5 members = 25 functions (see Section 4)
✔	Menu-driven interface using switch	main() has a switch block; sub-menus for each module
✔	Separate functions for logical activities	No business logic inside main(); all delegated to module functions
✔	Arrays of structures	MorseEntry entries[128], SessionLog logs[500], UserProfile users[50]
✔	String handling	strcpy, strcmp, strlen, strcat used throughout encode/decode and logging
✔	Proper input/output handling	scanf, fgets, printf, fprintf, fopen/fclose for file I/O
✔	Meaningful real-world data	ITU Morse code table, session timestamps, user profiles, prosigns
✔	Clear comments in code	Every function has a header comment block; inline comments for complex logic
✔	Individual contribution clearly shown	Each file/section tagged with member name; report section per member

Status	Requirement	How It Is Met
★	Pointers	Structures passed by pointer to all functions; pointer arithmetic in string ops
★	Sorting & Searching	Binary search (lookupCharacter), bubble sort (sortUsersByActivity), linear search
★	Nested structures	AlphabetSet contains an array of MorseEntry (nested structure array)
★	Input validation	validateMorseInput(), PIN format check, username uniqueness check
★	File I/O	Session logs and reports exported to .txt files; reloaded on next launch

7. Work Distribution & Timeline

7.1 Member Responsibilities

Member	Module	Structures	Functions	Additional Responsibilities
Member 1 (Module Lead M1)	Encoder / Decoder Engine	MorseEntry	loadMorseTable, encodeText, decodeMorse, validateMorseInput, lookupCharacter	Morse table data file, core algorithm design
Member 2 (Module Lead M2)	Session Logger	SessionLog	createSession, logTranslation, saveLogsToFile, loadLogsFromFile, displaySessionHistory	File I/O design, timestamp handling
Member 3 (Module Lead M3)	User Profile Manager	UserProfile	registerUser, loginUser, displayUserStats, updateUserStats, sortUsersByActivity	Authentication flow, PIN validation
Member 4 (Module Lead M4)	Multi-Alphabet Manager	AlphabetSet (nested MorseEntry)	initAlphabets, switchAlphabet, displayAlphabetTable, addCustomEntry, validateCoverage	Prosign table, multi-alphabet integration
Member 5 (Module Lead M5)	Analytics & Report Generator	AnalyticsReport	generateReport, printReportSummary, exportReportToFile, sortLogsByDate, searchLogByKeyword	Statistics engine, report file formatting

7.2 Suggested Development Timeline (3 Weeks)

Week	Tasks
Week 1 (Design)	• Define all structures and agree on interfaces • Set up shared code repository • Each member writes stub functions with comments • Morse table data preparation (Member 1)
Week 2 (Development)	• Implement all 25 functions independently • Unit test each module in isolation • Integrate modules through main() menu • File I/O integration (Member 2 leads)
Week 3 (Integration & Submission)	• Full system integration testing • Bug fixing and input validation • Write project report • Final code review, comments, and submission

8. Why Morse Code Translator & Logger?

When selecting a group project topic, we evaluated it against three criteria: **real-world relevance**, **technical depth**, and **equal divisibility** into five independent but connected modules. The Morse Code Translator & Logger excels on all three fronts.

8.1 Compared to Simple Alternatives

Criterion	Simple Output Programs	Morse Translator & Logger
Real-world use	■ None	✔ Ham radio, aviation, accessibility
Struct complexity	■ 1–2 trivial structs	✔ 5 distinct, meaningful structs
String handling	■ Minimal	✔ Extensive (encoding, file I/O, search)
File I/O	■ Rarely needed	✔ Session persistence, report export
Equal contribution	■ Hard to divide evenly	✔ Clear 5-module split
Sorting & searching	■ Not applicable	✔ Binary search, bubble sort on logs
Nested structures	■ Not used	✔ AlphabetSet contains MorseEntry[]

8.2 Learning Outcomes Achieved

- Practise defining and using C structs in a meaningful domain.
- Experience passing structs and arrays of structs to functions.
- Implement real search (binary search on the Morse table) and sort (logs by date).
- Handle file I/O: reading tables from disk, writing session logs and reports.
- Build a menu-driven switch interface that is extensible and well-organised.
- Collaborate as a team with clearly delineated module ownership.

9. Conclusion

The Morse Code Translator & Logger is an ideal choice for a 5-member console-based C project. It is grounded in a real and historically significant communication technology, naturally decomposes into five balanced modules, demands all the required C constructs (structs, arrays, string handling, file I/O, sorting, searching, and validation), and produces a result that is demonstrably useful in the real world. We are confident this project will allow every team member to make a meaningful, equal, and technically substantial contribution.